Inserting, Updating and Deleting Documents

Inserting Documents

- In this section, you will learn to insert new documents into a MongoDB collection. MongoDB collections provide a function named **insert()**, which is used to create a new document in a collection.

- The function is executed on the collection and takes the document to be inserted as an argument.

- This proves that, when a document **insert command** is executed, MongoDB will also create the given collection, if it does not exist already.

- MongoDB collections also provide the **insertMany()** function, which is a function specifically meant for inserting multiple documents into a collection.

- The **_id** field is the default ID of a Document in MongoDB. It is automatically tracked by the database. Each document must have a unique **_id** value. If we don't give an **_id**, then MongoDB will automatically generate one.

- When using **insertMany()**, it is possible for some documents to be successfully inserted, and other document insertions to fail.

Inserting Without _ID

- If we don't give an **_id**, then MongoDB will automatically generate one.

- While working on datasets, our documents will have unique fields that can be used as primary keys.

- Primary keys are the ones that can uniquely identify a record. MongoDB's auto generated keys are useful in terms of uniqueness, but they are meaningless in terms of the data the respective document represents.

- Also, these autogenerated keys are lengthy and thus tedious to type in or remember. Thus, we should use our own primary keys.

Deleting Using deleteOne()

- The function **deleteOne()** is used to delete a single document from a collection. It accepts a document representing a query condition.

- Upon successful execution, it returns a document containing **the total number of documents deleted (represented by the field deletedCount**) and whether the operation was confirmed (**given by the field acknowledged**). However, as the method deletes only one document, the value of deletedCount is always one.

- If the given query condition matches more than one document in the collection, only the first document will be deleted.

Deleting Using deleteMany()

- The **deleteMany()** function can delete many documents simultaneously. It must be provided with a query condition, and all the documents that match the given query will be removed.

- The output from the function will be an acknowledgement and the number of documents that're removed.

- Passing an empty query document is equivalent to not passing any filter, and thus, all the documents in the collection are matched.

- The query **{ "non_existent_field": "null"}** will match all the documents in a given collection. **This is because, in MongoDB a field that doesn't exist is considered null.** Thus, such a query shouldn't be used with **deleteMany()**.

Deleting Using findOneAndDelete()

The **findOneAndDelete()** function finds and deletes one document from the collection:

- It finds one document and deletes it.

- If more than one document is found, only the first one will be deleted.

- Once deleted, it returns the deleted document as a response.

- In the case of multiple document matches, the sort option can be used to influence which document gets deleted.

- Projection can be used to include or exclude fields from the document in response.

Exercise 5.02 - Deleting a Low Rated Movie - Page 227

Replacing Documents

- To replace a single document in a collection, MongoDB provides the method **replaceOne()**, which accepts a **query filter** and a **replacement document**. The function finds the document that matches the criteria and replaces it with the provided document.

- The function returns an acknowledgement on a successful replacement.

- If there is more than one document that matches the query, then only the first one will be replaced.

- The **_id** of the original document is retained in the replacement (new) document.

_ID Fields Are Immutable

- The **_id** of the original document is retained in the replacement (new) document.

- This is because **_id fields are immutable in MongoDB**. Immutable fields are like normal fields; however, once assigned with a value, their value cannot be changed again.

- The **_id** field serves as a unique identifier of a document and so should not be changed as long as the document exists. It isn't possible to change the **_id** field of a document using any method, including the **replaceOne()** function.

Upsert Using Replace

- **Upsert** is an operation that either **updates (replaces)** a document if it already exists, or **inserts (creates)** it if it doesn't exist. It is a portmanteau of the words **update** / **insert**.

- Executing an **upsert** command is faster and more efficient than executing a **replace** command to check all the results of the documents and then executing an **insert** command.

- The **upsert** command is achieved by setting **{ upsert: true }** in the **replaceOne()** function.

Replacing Using findOneAndReplace()

The **findOneAndReplace()** function does what its name indicates.
It has the following features:

- As the name indicates, it finds one document and replaces it.

- If more than one document is found matching the query, the first one will be replaced.

- A sort option can be used to influence which document gets replaced if more than one document is matched.

- By default, it returns the original document.

- If the flag **{returnNewDocument: true}** is set, the newly added document will be returned.

- Field projection can be used to include only specific fields in the document returned in response.

- By default, this function returns the title of the successfully replaced document as a response to the successful replacement. If we want the replaced document as a response, then we have to set the **{returnNewDocument: true}** flag.

Replace vs. Delete and Reinsert

- It is possible to replace a document using a combination of delete and insert, where you delete an existing document and insert a new one.

- This is achieved by using **findOneAndDelete()** and **insert()**. We delete the document using **findOneAndDelete()** and store the returned deleted document in a variable. We then modify and insert that document by using the **insert()** function.

Disadvantages of using Delete and Reinsert over Replace

- In the delete and insert method, **the data is transferred over the wire multiple times**. This involves the drivers or clients to parse the data in multiple stages. This will **slow down the overall performance**.

- When multiple clients are constantly reading and writing to your collections, **concurrency issues** may arise.

- Your database client or driver **may lose its connection to the database in the middle of two operations**. For example, the delete operation was successful but insertion could not happen.

The dedicated **replace functions, on the other hand, are effectively atomic** and are therefore **safe to use in concurrent environments**.

Updating a Document with updateOne()

- To update the fields of a single document in a collection, we can use the function **updateOne()**. This function, which is provided by MongoDB collections, accepts a query condition to find the record to be updated, and a document that specifies the field-level update expressions.

- Once the update is performed, it returns a detailed result in the form of a document, which indicates how many records were matched and how many records were updated.

- The **$set operator** that we used to update a field of a document can also be used to modify multiple fields of a document.

- It is important to know that, in an update operation, updating the same field multiple times is valid, irrespective of the field's value. THe meaning of this is that, **if we provide multiple update expressions for a field** in the **updateOne()** function, **then MongoDB will apply each of the update expressions and the field will be updated with the value of the last update expression**.

- If the **query** in the **updateOne()** function matches more than 1 document, then only the first document will be updated.

- It is possible to perform **upsert operation** using **updateOne()** by including the **{upsert: true}** flag.

Updating a Document with findOneAndUpdate()

- MongoDB also provides the **findOneAndUpdate()** function, which is capable of doing everything that **updateOne()** does with a few additional features.

- **findOneAndUpdate()** needs at least two arguments where the first one is a **query condition** to find the document to be modified and the **second one is the update expression**. By default, it returns the old document in the response.

- It can be set to return the new document in the response by using the **{returnNewDocument: true}** flag.

Sorting to Find A Document

- The **findOneAndUpdate()** function provides an additional option to sort the matching documents in a specific order. Using the sort option, you can **influence which document is selected** for the modification.

- The sort option is specified as a field under the optional third argument of the **findOneAndUpdate()** function. The value of the sort field **must be a document containing valid sort expressions**.

- The **{sort: -1}** flag is set to get the document with the largest **_id**. The **{sort: 1}** flag is set to get the document with the smallest **_id**. Note that **-1** in a sorting context means **descending order** while **1** means **ascending order**.

Exercise 5.03: Updating the IMDb and Tomatometer Rating - Page 254

Updating Multiple Documents with updateMany()

- MongoDB provides the **updateMany()** function, which updates multiple documents at a time. It takes two mandatory arguments.

- The first argument is the **query condition**, and the second is the **update expression**. The third argument, which is optional, is used to provide **miscellaneous options**.

Some Important Points Regarding updateOne(), updateMany() and findOneAndUpdate()

- None of the update functions allows you to change the **_id** field.
- The order of the fields in a document is always maintained, except when the update includes renaming a field. However, the **_id** field will always appear first.
- Update operations are atomic on a single document. A document cannot be modified until another process has finished updating it.
- All of the update functions support upsert. To execute an upsert command, **{ upsert : true}** needs to be passed as an option.

Update Operators

- Syntax of an update operator: **{ <update_operator>: {field_1: value_1, …} }**
- The above update expression can be used to update many different fields with the same operator.
- If we want to update each field with a different operator, then we need to have an update expression with multiple update operators, each separated by a comma. Eg - **{ <update operator 1>: {<field11> : <value11>, ... }, <update operator 2>: {<field21> : <value21>, ... }, ..., }**.

The $set Operator

- The **$set** operator is used to set the values of fields in a document.
- It can be easily used to set values of any type of field or add new fields in a document.
- It takes a document that contains pairs of field names and their new values. If the given field is not already present, it will be created.

The $inc Operator

- The increment operator (**$inc**) is used to increment the value of a numeric field by a specific number.
- The operator accepts a document containing pairs of a field name and a number.
- Given a positive number, the value of the field will be incremented and if a negative number is provided, the value will be decremented.
- This operator can only be used with numeric fields.
- If a non-existent numeric field is attempted to be updated using **$inc**, then **$inc** will create that field with the update value.

The $mul Operator

- The multiplication (**$mul**) operator is used to multiply the value of a numeric field by the given number.

- The operator accepts a document containing pairs of field names and numbers and can only be used on numeric fields.

- If a non-existent numeric field is attempted to be updated using **$mul**, then **$mul** will create that field with the value set to 0.

The $rename Operator

- The **$rename** operator is used to rename fields. The operator accepts a document containing pairs of field names and their new names.

- If the field is not already present in the document, the operator ignores it and does nothing.

- The provided field and its new name must be different. If they're the same, the operation fails with an error.

- If a document already contains a field with the provided new name, the existing field will be removed.

- Using this operator, we can also move a field to a nested document or move it to the root document from the nested document.

The $currentDate Operator

- The operator **$currentDate** is used to set the value of a given field as the current date or timestamp. If the field is not present already, it will be created with the current date or timestamp value.

- Providing a field name with a value of **true** will insert the current date as a **Date**. Alternatively, a **$type** operator can be used to explicitly specify the value as a **date** or **timestamp**.

- If the value is specified as a **date**, then an **ISODate()** object is assigned. If the value is specified as a **timestamp**, then a **Timestamp()** object is assigned.

Removing Fields ($unset)

- The **$unset** operator removes given fields from a document. The operator accepts a document containing pairs of field names and values and removes all the given fields from the matched document.

The $setOnInsert Operator

- The operator **$setOnInsert** is similar to **$set;** however, it only sets the given fields when an insert happens during an upsert operation. It has no impact when the upsert operation results in the update of existing documents.